

static_sized_range

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Proposed change

Abstract

This paper introduces `static_sized_range`, a refinement of `sized_range` for ranges whose sizes are known at compile time.

It allows detecting and retrieving a range's size as a constant expression through a new variable template `range_static_size_v` with a similar naming style to `range_size_t`.

This enables compile-time reasoning about range sizes and improves `constexpr` support and optimization opportunities in range adaptors and algorithms.

Revision history

R0

Initial revision.

Discussion

The Ranges library initially attempted to identify ranges with compile-time known sizes using an exposition-only concept, `tiny-range`:

```
template<auto> struct require-constant;                                // exposition only

template<class R>
concept tiny-range =                                                // exposition only
    sized_range<R> &&
    requires { typename require-constant<remove_reference_t<R>::size()>; } &&
    (remove_reference_t<R>::size() <= 1);
```

However, this approach relied on the presence of a static member `size()` and excluded many types, such as `span<int, 1>`. There was no general mechanism in the language to allow generic code to verify whether `ranges::size(r)` could be evaluated as a constant expression.

The acceptance of [P2280](#) provided a new opportunity: by modifying the rules of constant evaluation, it allows expressions involving references to unknown objects to be considered valid constant expressions when the result does not depend on the object's identity.

This makes `ranges::size(r)` more generally usable at compile time and allows defining a concept like `static_sized_range` in a simpler and more intuitive way. Additionally, the C++26 `simd` specification heavily relies on this language enhancement: its wording repeatedly checks that `ranges::size(r)` is a constant expression to ensure, at compile time, the correctness of vector sizes.

However, these compile-time checks and optimizations should not be limited to `simd`. Introducing the `static_sized_range` concept is therefore worthwhile, as it enables compile-time reasoning about range sizes and broader use in generic programming.

Design

New concept and variable template

This proposal introduces two related entities: the `static_sized_range` concept and the `range_static_size_v` variable template. The `static_sized_range` concept refines `sized_range` by further requiring that `ranges::size(r)` be evaluable as a constant expression. It is defined as follows:

```
template<class T>
concept static_sized_range =
    sized_range<T> && requires(T& t) { cw<ranges::size(t)>; };
```

Here, `cw` enables the formation of a `constant_wrapper` introduced in [P2781](#) from an expression known to be a constant expression, allowing the concept to directly check whether `ranges::size(r)` can be evaluated at compile time, as enabled by [P2280](#).

Once such a range is identified, its compile-time size can be retrieved through a variable template:

```
template<static_sized_range T>
constexpr auto range_static_size_v =
    decltype([](T& t) { return cw<ranges::size(t)>; }(declval<T&>()))::value;
```

The definition relies on the same mechanism: it promotes the result of `ranges::size(r)` to a type via `cw`, and then extracts the value from that type. This effectively turns a compile-time constant into a type-level entity, allowing the variable template to retrieve the constant directly through ordinary deduction.

Together, `static_sized_range` and `range_static_size_v` provide a minimal yet general mechanism for compile-time reasoning about range extents:

```
static_assert(ranges::static_sized_range<array<int, 3>>);
static_assert(ranges::range_static_size_v<array<int, 3>> == 3);

static_assert(ranges::static_sized_range<span<int, 5>>);
static_assert(ranges::range_static_size_v<span<int, 5>> == 5);

static_assert(ranges::static_sized_range<ranges::single_view<int>>);
static_assert(ranges::range_static_size_v<ranges::single_view<int>> == 1);

static_assert(ranges::static_sized_range<ranges::empty_view<int>>);
static_assert(ranges::range_static_size_v<ranges::empty_view<int>> == 0);

static_assert(!ranges::static_sized_range<span<int>>);
static_assert(!ranges::static_sized_range<vector<int>>);
static_assert(!ranges::static_sized_range<optional<int>>);
static_assert(ranges::range_static_size_v<string> == 42); // ill-formed: constraints not satisfied
```

Enhancements to integer-class type (LWG [4409](#))

Although `range_static_size_v<R>` exposes the size of a `static_sized_range` at compile time, expressions such as `(range_static_size_v<R> >= 1)` may not be constant expressions, since `ranges::size(r)` can return an integral-class type lacking `constexpr` comparison or arithmetic operators.

LWG [4409](#) highlights this limitation, noting that such types may behave like integers at runtime but are not necessarily usable in constant expressions.

However, it is natural to expect that integer-class types should support compile-time operations like built-in integers. To align with this expectation, it is desirable to clarify the wording for any integer-class type defined in [\[iterator.concept.winc\]](#), ensuring well-defined `constexpr` behaviors.

Enhancements to `ref_view`

Currently, `ref_view<R>::size()` simply forwards to the underlying range via `*r_`:

```
constexpr auto size() const requires sized_range<R>
{ return ranges::size(*r_); }
```

Even if `R` satisfies `static_sized_range`, this expression is still not a constant expression. As a result, for example, `ref_view<array<int, 42>>` does not model `static_sized_range`.

This can be fixed by having `size()` return `range_static_size_v<R>` when `R` models `static_sized_range`, while keeping the original behavior for other ranges:

```
constexpr auto size() const requires sized_range<R> {
    if constexpr (static_sized_range<R>)
        return range_static_size_v<R>;
    else
        return ranges::size(*r_);
}
```

Similarly, `empty()` should receive similar treatment so that `ranges::empty` on `ref_view` can also be a constant expression. The effect of this change can be seen in the following example:

```
array a{1, 2, 3, 4, 5};
auto r = a | views::transform([](int i) { return i * i; })
          | views::reverse;

static_assert(ranges::size(r) == 5); // ok
static_assert(!ranges::empty(r));   // ok
static_assert(ranges::static_sized_range<decltype(r)>); // ok
static_assert(ranges::range_static_size_v<decltype(r)> == 5); // ok
```

Enhancements to front/back across array/span/view_interface

For `view_interface`, the `front()` and `back()` members can benefit from `static_sized_range`. If a derived view models `static_sized_range`, calls on an empty range can be rejected at compile time.

Similarly, `array<T, N>` and `span<T, N>` with a fixed extent can take advantage of this: when `N == 0`, invoking `front()` or `back()` can be diagnosed at compile time, providing safety without relying on runtime *Hardened preconditions* added in [P3471](#).

Enhancements to join_view (LWG [4401](#))

LWG [4401](#) observes that `join_view` is not considered a `sized_range`, even though in certain cases — such as when the outer range is sized and the inner range has a fixed size — its total size can be determined.

With the introduction of `static_sized_range` and `range_static_size_v`, this limitation can be addressed more generally: when the inner range models `static_sized_range`, the size of a `join_view` can be computed as `ranges::size(base_) * range_static_size_v<InnerRng>`.

This effectively makes `ranges::join_view<span<array<int, 3>, 4>>>` a `static_sized_range` with a static size of 12.

Enhancements to lazy_split_view (LWG [3855](#), [4108](#))

LWG [3855](#) notes that *tiny-range* is limited and not fully general. Using `static_sized_range` and `range_static_size_v`, we can rewrite *tiny-range* with a concept that correctly captures ranges whose size is known at compile time, and a range with `range_static_size_v<R> <= 1` naturally satisfies the intended semantics of *tiny-range*.

This means that `lazy_split_view` can now take an `array` or `span` of size 1 or 0 as the patterns, which is a nice enhancement:

```
array a{42};
auto r1 = views::istream<int>(in1) | views::lazy_split(a); // ok
auto r2 = views::istream<int>(in2) | views::lazy_split(array{42}); // ok
auto r3 = views::istream<int>(in3) | views::lazy_split(span{a}); // ok
```

LWG [4108](#) observes that `lazy_split_view` cannot provide `size()` for certain valid cases. With `static_sized_range`, this can be improved: when the underlying range is sized and the pattern is statically empty, `lazy_split_view` can conditionally provide `size()` computed as `ranges::size(base_)`.

Enhancement to span (LWG [4397](#), [4404](#))

LWG [4397](#) concerns constructing a `span` from a statically sized range. The standard currently enforces this with *Hardened Preconditions* at runtime to reject ranges whose size does not match the fixed extent. Applying `static_sized_range` allows this requirement to be expressed at compile time, which is a useful enhancement.

LWG [4404](#) deals with class template argument deduction (CTAD) for `span(R&&)`. When the underlying range has a statically known size, the current standard does not uniformly propagate this information through CTAD due to language rule before [P2280](#). With the newly `static_sized_range`, the compiler can conditionally treat such `span` constructions as statically sized, enabling more precise compile-time checks and better integration with generic code.

Enhancement to `simd/inplace_vector` wording (LWG [4396](#))

In C++26 `simd`, vector sizes must be known at compile time so that the compiler can generate correct SIMD instructions. The standard currently expresses this requirement as "If `ranges::size(r)` is a constant expression," which is verbose and not easily reusable in generic code.

With the introduction of `static_sized_range`, this check can be simplified to "If `R` models `static_sized_range`". This formulation is concise, clearly expresses the intent, and can be applied consistently across the standard library. Combined with `range_static_size_v`, it provides a uniform mechanism for compile-time size reasoning.

Note that the similar wording in LWG [4396](#) can also be simplified with `static_sized_range`, making the intent explicit and consistent with `simd`.

Enhancement to `ranges::min/max/minmax`

Those algorithms specify as a *Preconditions* that the input range must not be empty.

With the introduction of `static_sized_range`, these preconditions can be verified at compile time. For example, invoking `ranges::min` on `array<int, 0>` could be statically rejected, improving diagnostic clarity and program safety by turning a runtime undefined behavior into a compile-time error.

Additionally, `static_sized_range` provides opportunities for compile-time optimizations, with many more potential enhancements throughout the library. For example, `ranges::equal` between two ranges of different static sizes can be evaluated to false at compile time, reducing unnecessary instantiations and runtime checks.

Compile-time size considerations for `optional/inplace_vector`

After [P3168](#), `optional` could in principle be treated as a *tiny-range*, since its maximum size is one. If it were considered *tiny-range*, `lazy_split_view` could use an `optional` as the pattern for input ranges:

```
auto r = views::istream<int>(in) | views::lazy_split(optional{42});
```

However, *tiny-range* is intended to have a fully known, precise size at compile time to support efficient code generation, which `optional` does not provide. For this reason, the author does not adopt such a treatment.

Additionally, similar to `optional`, `inplace_vector` also has a statically known maximum size. In theory, this could be used to enhance range adaptors such as `reserve_hint` member introduced in [P2846](#): for example, a `join_view` of a range of `optional` could have `reserve_hint` returning `ranges::size(base_) * 1`, and `ranges::size(base_) * N` for range of `inplace_vector<T, N>`. However, this is a very specialized optimization and is not pursued here.

In theory, calls to `front()`, `back()`, or `pop_back()` on an `inplace_vector` with zero capacity could be rejected at compile time. However, this is an extremely limited case and does not justify adding such checks.

Proposed change

This wording is relative to [Lastest Working Draft](#).

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define __cpp_lib_ranges_static_sized_range 2026XXL // freestanding, also in <ranges>
```

2. Modify 23.2.4 [\[sequence.reqmts\]](#) as indicated:

```
a.assign_range(rg)
```

-60- Result: `void`

-61- Mandates: `assignable_from<T&, ranges::range_reference_t<R>>` is modeled. For `inplace_vector`, if `R` models `ranges::static_sized_range` ~~`ranges::size(rg)` is a constant expression~~ then `ranges::range_static_size_v<R>` ~~`ranges::size(rg)`~~ $\leq$ `a.max_size()`.

[...]

`a.front();`

-71- Result: `reference`; `const_reference` for constant `a`.

-?- Mandates: For array, `a.empty()` is false.

-72- Hardened preconditions: `a.empty()` is false.

[...]

`a.back();`

-75- Result: `reference`; `const_reference` for constant `a`.

-?- Mandates: For array, `a.empty()` is false.

-76- Hardened preconditions: `a.empty()` is false.

3. Modify 23.3.16.2 [\[inplace_vector.cons\]](#) as indicated:

```
template<container-compatible-range<T> R>
constexpr inplace_vector(from_range_t, R&& rg);
```

-?- Mandates: If `R` models `ranges::static_sized_range` ~~`ranges::size(rg)` is a constant expression~~ then `ranges::range_static_size_v<R>` ~~`ranges::size(rg)`~~ $\leq$ `N`.

-9- Effects: Constructs an `inplace_vector` with the elements of the range `rg`.

-10- Complexity: Linear in `ranges::distance(rg)`.

4. Modify 23.7.2.2.2 [\[span.cons\]](#) as indicated:

```
template<class R> constexpr explicit(extent != dynamic_extent) span(R&& r);
```

-?- Mandates: If `extent` is not equal to `dynamic_extent` and `R` models `ranges::static_sized_range`, then `ranges::range_static_size_v<R> == extent`.

-16- Constraints: Let `U` be `remove_reference_t<ranges::range_reference_t<R>>`. [...]

5. Modify 23.7.2.2.3 [\[span.deduct\]](#) as indicated:

```
template<class R>
span(R&&) -> see below span<remove_reference_t<ranges::range_reference_t<R>>>;
```

-2- Constraints: `R` satisfies `ranges::contiguous_range`.

-?- Remarks: Let `T` denote the type `remove_reference_t<ranges::range_reference_t<R>>`. The deduced type is `span<T, static_cast<size_t>(ranges::range_static_size_v<R>>)` if `R` models `ranges::static_sized_range`, otherwise `span<T>`.

6. Modify 23.7.2.2.6 [\[span.elem\]](#) as indicated:

```
constexpr reference front() const;
```

-?- Mandates: If `extent` is not equal to `dynamic_extent`, then `empty()` is false.

-6- Hardened preconditions: `empty()` is false.

[...]

constexpr reference back() const;

-?- Mandates: If extent is not equal to dynamic extent, then empty() is false.

-9- Hardened preconditions: empty() is false.

7. Modify 24.3.4.4 [\[iterator.concept.winc\]](#) as indicated:

-9- All integer-class types model regular ([\[concepts.object\]](#)) and three_way_comparable<strong_ordering> ([\[cmp.concept\]](#)).

-?- Operations on integer-class type shall be usable in constant expressions.

8. Modify 25.2 [\[ranges.syn\]](#) as indicated:

```
// mostly freestanding
#include <compare>           // see \[compare.syn\].
#include <initializer_list>   // see \[initializer.list.syn\].
#include <iterator>           // see \[iterator.synopsis\].

namespace std::ranges {
    [...]
    // \[range.sized\], sized ranges
    template<class>
        constexpr bool disable_sized_range = false;

    template<class T>
        concept approximately_sized_range = see below;

    template<class T>
        concept sized_range = see below;

    template<class T>
    concept static_sized_range = see below;

    template<static sized range T>
    constexpr auto range_static_size v = see below;
    [...]
}
```

9. Add **25.4.? Static sized ranges** [\[range.static.sized\]](#) after 25.4.4 [\[range.sized\]](#) as indicated:

-1- The static sized_range concept refines sized_range with the requirement that the number of elements in the range can be determined in compile-time using ranges::range_static_size v.

```
template<class T>
concept static sized_range =
sized_range<T> && requires(T& t) { cw<ranges::size(t)>; };

auto get-size-cw(auto& t) // exposition only
-> decltype(cw<ranges::size(t)>);

template<static sized range T>
constexpr auto range_static_size v =
decltype(get-size-cw(declval<T&>()))::value;
```

10. Modify 25.5.3.2 [\[view.interface.members\]](#) as indicated:

```
constexpr decltype(auto) front() requires forward_range<D>;
constexpr decltype(auto) front() const requires forward_range<const D>;
```

-?- Mandates: Let R be decltype(derived()). If R models ranges::static sized_range, then ranges::range_static_size v<R> > 0.

-1- Hardened preconditions: !empty() is true.

-2- Effects: Equivalent to: return *ranges::begin(derived());

constexpr decltype(auto) back() requires bidirectional_range<D> && common_range<D>;
 constexpr decltype(auto) back() const
 requires bidirectional_range<const D> && common_range<const D>;

-?- Mandates: Let R be decltype(derived()). If R models ranges::static sized range, then ranges::range static size v<R> > 0.

-3- Hardened preconditions: !empty() is true.

11. Edit 25.7.6.2 [\[range.ref.view\]](#) as indicated:

```
namespace std::ranges {
  template<range R>
    requires is_object_v<R>
    class ref_view : public view_interface<ref_view<R>> {
    [...]
    public:
    [...]
      constexpr bool empty() const
        requires requires { ranges::empty(*r_); } {
        if constexpr (static sized range<R>)
        return range_static_size v<R> == 0;
        else
        return ranges::empty(*r_);
      }

      constexpr auto size() const requires sized_range<R> {
        if constexpr (static sized range<R>)
        return range_static_size v<R>;
        else
        return ranges::size(*r_);
      }
    [...]
  };
  [...]
}
```

12. Modify 25.7.14.2 [\[range.join.view\]](#) as indicated:

```
namespace std::ranges {
  template<input_range V>
    requires view<V> && input_range<range_reference_t<V>>>
    class join_view : public view_interface<join_view<V>> {
    [...]
    public:
    [...]
      constexpr auto size()
      requires sized_range<V> &&
      static sized range<InnerRng> {
      using CT = common_type_t<range_size_t<V>, range_size_t<InnerRng>>;
      return CT(ranges::size(base_)) * CT(range_static_size v<InnerRng>);
      }

      constexpr auto size() const
      requires sized_range<const V> &&
      static sized range<range_reference_t<const V>> {
      using InnerConstRng = range_reference_t<const V>;
      using CT = common_type_t<range_size_t<const V>, range_size_t<InnerConstRng>>;
      return CT(ranges::size(base_)) * CT(range_static_size v<InnerConstRng>);
      }
    [...]
  };
  [...]
}
```

13. Modify 25.7.15.2 [\[range.join.with.view\]](#) as indicated:

```
namespace std::ranges {
  [...]
  template<input_range V, forward_range Pattern>
    requires view<V> && input_range<range_reference_t<V>>
    && view<Pattern>
    && concatable<range_reference_t<V>, Pattern>
    class join_with_view : public view_interface<join_with_view<V, Pattern>> {
```



```

[...]  

public:  

[...]  

constexpr auto size()  

    requires sized_range<V> &&  

           sized_range<Pattern> &&  

           static sized_range<InnerRng> {  

    using CT = common_type_t<  

        range_size_t<V>, range_size_t<InnerRng>, range_size_t<Pattern>>;  

    const auto base_size = ranges::size(base_);  

    if (base_size == 0)  

        return CT(0);  

    return CT(base_size) * CT(range_static_size v<InnerRng>) +  

        CT(base_size - 1) * CT(ranges::size(pattern));  

    }  

  

constexpr auto size() const  

    requires sized_range<const V> &&  

           sized_range<const Pattern> &&  

           static sized_range<range reference t<const V>> {  

    using InnerConstRng = range reference t<const V>;  

    using CT = common_type_t<  

        range_size_t<const V>, range_size_t<InnerConstRng>, range_size_t<const Pattern>>;  

    const auto base_size = ranges::size(base_);  

    if (base_size == 0)  

        return CT(0);  

    return CT(base_size) * CT(range_static_size v<InnerConstRng>) +  

        CT(base_size - 1) * CT(ranges::size(pattern));  

    }  

};  

[...]  

}

```

14. Modify 25.7.16.2 [\[range.lazy.split.view\]](#) as indicated:

```

namespace std::ranges {  

template<auto> struct require_constant; // exposition only  

  

template<class R>  

concept tiny_range = // exposition only  

    static sized_range<R> &&  

requires (typename require_constant<remove_reference_t<R>::size()>;) &&  

    (range_static_size v<R>remove_reference_t<R>::size() <= 1);  

  

template<input_range V, forward_range Pattern>  

    requires view<V> && view<Pattern> &&  

           indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&  

           (forward_range<V> || tiny_range<Pattern>)  

class lazy_split_view::view_interface<lazy_split_view<V, Pattern>> {  

[...]  

    constexpr auto size()  

        requires sized_range<V> &&  

               static sized_range<Pattern> &&  

               (range_static_size v<Pattern> == 0)  

    { return ranges::size(base_); }  

  

    constexpr auto size() const  

        requires sized_range<const V> &&  

               static sized_range<Pattern> &&  

               (range_static_size v<Pattern> == 0)  

    { return ranges::size(base_); }  

};  

[...]  

}

```

15. Modify 25.7.16.5 [\[range.lazy.split.inner\]](#) as indicated:

```
constexpr inner_iterator& operator++();
```

-5- Effects: Equivalent to:

```

incremented_ = true;  

if constexpr (!forward_range<Base>) {  

    if constexpr (range_static_size v<Pattern>Pattern::size() == 0) {

```



```

        return *this;
    }
}
++i_.current;
return *this;

```

16. Modify 25.7.17.2 [\[range.split.view\]](#) as indicated:

```

namespace std::ranges {
    template<forward_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
        indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
    class split_view : public view_interface<split_view<V, Pattern>> {
    [...]
        constexpr auto size()
            requires sized_range<V> &&
                static sized_range<Pattern> &&
                (range_static_size v<Pattern> == 0) {
            return ranges::size(base_);
        }

        constexpr auto size() const
            requires sized_range<const V> &&
                static sized_range<Pattern> &&
                (range_static_size v<Pattern> == 0) {
            return ranges::size(base_);
        }
    };
    [...]
}

```

17. Modify 26.8.9 [\[alg.min.max\]](#) as indicated:

```

template<class T>
    constexpr T min(initializer_list<T> r);
[...]
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    range_value_t<R>
    ranges::min(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

    -?- Mandates: If R models ranges::static_sized_range, then
    ranges::range_static_size v<R> > 0.

    -5- Preconditions: ranges::distance(r) > 0. [...]

[...]

template<class T>
    constexpr T max(initializer_list<T> r);
[...]
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    range_value_t<R>
    ranges::max(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

    -?- Mandates: If R models ranges::static_sized_range, then
    ranges::range_static_size v<R> > 0.

    -13- Preconditions: ranges::distance(r) > 0. [...]

[...]

template<class T>
    constexpr pair<T, T> minmax(initializer_list<T> t);
[...]
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    ranges::minmax_result<range_value_t<R>>
    ranges::minmax(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

-?- Mandates: If R models ranges::static sized range, then ranges::range static size v<R> > 0.

-21- Preconditions: ranges::distance(r) > 0. [...]

18. Modify 29.10.7.2 [simd.ctor] as indicated:

```
template<class R, class... Flags>
constexpr basic_vec(R&& r, flags<Flags...> = {});
template<class R, class... Flags>
constexpr basic_vec(R&& r, const mask_type& mask, flags<Flags...> = {});
```

-12- Let mask be mask_type(true) for the overload with no mask parameter.

-13- Constraints:

(13.1) — R models ranges::contiguous_range and ranges::static sized_range,

~~(13.2) — ranges::size(r) is a constant expression,~~

(13.3) — ranges::range static size v<R>~~ranges::size(r)~~ is equal to size(), and

(13.4) — ranges::range_value_t<R> is a vectorizable type and satisfies ~~explicitly-convertible-to<T>~~.

[...]

```
template<class R, class... Ts>
basic_vec(R&& r, Ts...) -> see below;
```

-17- Constraints:

~~(17.1) — R models ranges::contiguous_range and ranges::static sized_range;~~
~~and~~

~~(17.2) — ranges::size(r) is a constant expression.~~

-18- Remarks: The deduced type is equivalent to vec<ranges::range_value_t<R>, static_cast<simd-size-type>(ranges::range static size v<R>~~ranges::size(r)~~)>.

19. Modify 29.10.8.6 [simd.loadstore] as indicated:

```
template<class V = see below, ranges::contiguous_range R, class... Flags>
requires ranges::sized_range<R>
constexpr V unchecked_load(R&& r, flags<Flags...> f = {});
[...]
```

```
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
constexpr V unchecked_load(I first, S last, const typename V::mask_type& mask,
                           flags<Flags...> f = {});
```

-1- Let [...]

-2- Mandates: If R models ranges::static sized_range~~ranges::size(r) is a constant expression~~ then ranges::range static size v<R>~~ranges::size(r)~~ ≥ V::size().

[...]

```
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
requires ranges::sized_range<R>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r, flags<Flags...> f = {});
[...]
```

```
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
                               const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
```

-11- Let [...]

-22- *Mandates:* If `R` models `ranges::static sized range`~~`ranges::size(r)` is a constant expression~~ then `ranges::range static size v<R>`~~`ranges::size(r)`~~ $\geq$ `simd-size-v<T, Abi>`.